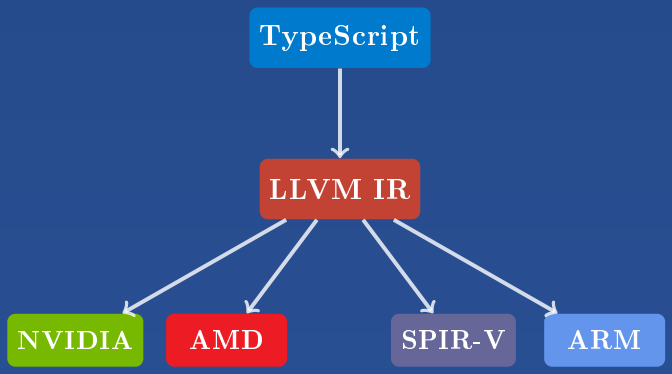
Velislav Simov Karastoychev

Programming LLVM IR with TypeScript

A Practical Handbook for Cross-Platform and Multi-Architecture Code Generation



Programming LLVM IR with TypeScript

Velislav Karastoychev

Founder of Euriklis LTD
Co-founder of One Agent LTD

vskarastoychev@gmail.com

https://github.com/VelislavKarastoychev/euriklis-llvm-ir

November 29, 2025

Preface

The purpose of this handbook is to provide a comprehensive explanation of how to use the `@euriklis/llvm-ir` library for programmatic generation of LLVM intermediate representation targeting CPUs and GPUs from TypeScript and JavaScript.

We have chosen a pedagogical approach that emphasizes **learning through examples**. This methodology is particularly well-suited for a handbook intended as a practical guide. Rather than starting with abstract theory, we demonstrate concepts through working code that readers can immediately understand, modify, and experiment with. Each example builds upon previous ones, gradually introducing complexity while maintaining clarity.

However, given the inherent theoretical complexity of LLVM IR, compiler design, and heterogeneous computing, we also provide comprehensive theoretical foundations. The handbook includes formal LLVM definitions, detailed explanations of all instructions in the intermediate language, memory models, type systems, and architecture-specific intrinsics. This dual approach—practical examples supported by rigorous theory—ensures the handbook serves both as a tutorial for beginners and as a reference for experienced developers.

Acknowledgments

I would like to express my deep gratitude to ***Peter Sveshtarov*** for the fruitful conversations and precious guidance, and for all the time spent in discussions with him, which have left a bright mark on the understanding and conceptual constructions of such projects. A large part of his ideas and directions are reflected in this project.

Contents

Part I

Introduction

Chapter 1

What is LLVM IR and Why Is It Useful?

1.1 The Compiler Problem

Traditional compilers are monolithic: each language needs its own complete toolchain targeting every architecture. If you have M languages and N architectures, you need $M \times N$ implementations. This becomes unsustainable as both language diversity and hardware complexity grow.

LLVM solves this by introducing an **intermediate representation (IR)**—a stable, language-independent, architecture-independent format that sits between high-level source code and low-level machine code. Instead of $M \times N$ implementations, you need M front-ends (languages $\rightarrow$ IR) and N back-ends (IR $\rightarrow$ machine code), requiring only $M + N$ components.

1.2 What is LLVM IR?

LLVM IR is a low-level programming language that resembles assembly but with several critical differences:

- **Strongly typed**: Every value has an explicit type (`i32`, `float`, `ptr`, etc.)
- **Platform-independent**: Same IR works for x86, ARM, GPU, WebAssembly
- **SSA form**: Every variable is assigned exactly once (enables aggressive optimizations)
- **Infinite registers**: Virtual registers with `%` prefix (backend maps to physical registers)
- **Explicit control flow**: Basic blocks and explicit branches instead of goto

Example LLVM IR function:

```
define i32 @add(i32 %a, i32 %b) {
entry:
  %result = add i32 %a, %b
  ret i32 %result
}
```

This is:

- Human-readable (unlike binary machine code)
- Optimizable (unlike high-level source code)
- Portable (unlike assembly language)

1.3 Static Single Assignment (SSA) Form

SSA is a property where each variable is assigned exactly once and every use refers to a specific definition. This makes data flow explicit and enables powerful optimizations.

Non-SSA code:

```
x = 1
x = x + 2  // x is assigned twice (not SSA)
```

SSA code:

```
%x1 = 1
%x2 = add i32 %x1, 2  // Each value has unique name
```

Benefits:

- Constant propagation: `%x2` is provably 3
- Dead code elimination: Unused definitions can be safely removed
- Register allocation: Clear lifetime of each value

1.4 Basic Blocks and Control Flow

LLVM organizes code into **basic blocks**—straight-line sequences of instructions with no branches except at the end:

```
define i32 @max(i32 %a, i32 %b) {
entry:
  %cmp = icmp sgt i32 %a, %b  // Compare: a > b?
  br i1 %cmp, label %if.then, label %if.else

if.then:
  ret i32 %a

if.else:
  ret i32 %b
}
```

Each basic block:

1. Has a unique label (`entry`, `if.then`, `if.else`)
2. Contains sequential instructions (no control flow in the middle)
3. Ends with a **terminator** (`ret`, `br`, `switch`)

1.5 The PHI Instruction

When control flow merges, SSA form requires a special instruction to select the correct value:

```
define i32 @abs(i32 %x) {
entry:
  %is_neg = icmp slt i32 %x, 0
  br i1 %is_neg, label %negate, label %done

negate:
  %neg = sub i32 0, %x
  br label %done

done:
  %result = phi i32 [ %x, %entry ], [ %neg, %negate ]
```

```
  ret i32 %result
}
```

The `phi` instruction says: "If we came from `%entry`, use `%x`; if we came from `%negate`, use `%neg`."

1.6 Modules, Functions, and Targets

LLVM IR is organized hierarchically:

- **Module**: Top-level container (one per compilation unit)
- **Functions**: Named code units with parameters and return types
- **Basic blocks**: Control flow nodes within functions
- **Instructions**: Individual operations (add, load, store, etc.)

Each module specifies its **target**:

```
target triple = "x86_64-unknown-linux-gnu"
target datalayout = "e-m:e-p270:32:32-i64:64-f80:128-n8:16:32:64-S128"
```

The triple format is `architecture-vendor-os`:

- `x86_64-apple-darwin` - macOS on Intel
- `aarch64-unknown-linux-gnu` - Linux on ARM (Apple Silicon, AWS Graviton)
- `nvptx64-nvidia-cuda` - NVIDIA GPUs
- `amdgcn-amd-amdhsa` - AMD GPUs
- `wasm32-unknown-unknown` - WebAssembly

1.7 Why LLVM IR Is Useful

1. **Write once, run anywhere**: Generate IR once, compile to x86, ARM, GPU, WASM
2. **Optimization**: LLVM's optimizer works on IR (loop unrolling, vectorization, inlining)
3. **Tooling**: IR can be analyzed, transformed, interpreted, or JIT-compiled
4. **Incremental compilation**: Modules can be linked together
5. **Language experimentation**: Build new languages without writing a full compiler backend

This handbook shows how to generate LLVM IR from TypeScript, unlocking these benefits for the JavaScript ecosystem.

Chapter 2

Why LLVM for the TypeScript and JavaScript Ecosystem?

2.1 The Rise of Performance-Critical JavaScript

JavaScript and TypeScript have evolved beyond web browsers. They now power:

- **Scientific computing**: TensorFlow.js, GPU.js, numerical libraries
- **Machine learning**: Neural network training, inference engines
- **Computer graphics**: Three.js, Babylon.js, game engines
- **Data processing**: Stream processing, ETL pipelines, analytics
- **Blockchain**: Smart contract VMs, cryptographic operations

These workloads hit JavaScript's performance ceiling. While V8, SpiderMonkey, and JavaScriptCore have excellent JIT compilers, they cannot compete with ahead-of-time compiled, hardware-specific code for compute-intensive tasks.

2.2 Why Not Just Use C++ or Rust?

The traditional approach is to write performance-critical code in C++/Rust and bind it to JavaScript via Node.js addons, WebAssembly, or FFI. This works but introduces friction:

1. Write core algorithm in C++/Rust
2. Compile for multiple platforms (Windows, macOS, Linux, maybe mobile)
3. Bind to JavaScript with FFI
4. Debug across language boundaries

This workflow has friction:

- **Separate toolchains**: npm + cargo/cmake + wasm-pack
- **Build complexity**: native compilation per platform
- **Type mismatches**: JavaScript objects $\leftrightarrow$ C structs
- **Limited GPU access**: WebGPU is new; CUDA requires native code

2.3 LLVM as the Universal Backend

LLVM provides a *universal code generation backend*:

1. **Write code generation logic in TypeScript** (type-safe, testable, version-controlled with npm)

2. **Generate LLVM IR** (platform-independent intermediate representation)
3. **Compile to any architecture**: x86, ARM, CUDA, AMD, WebAssembly—from the same IR
4. **Leverage LLVM's optimizer**: Auto-vectorization, loop unrolling, inlining—for free

Example workflow:

```
// TypeScript
import { LLVMCodegen } from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("my_module");
// ... build IR programmatically ...

const ir = codegen.toString();
// Emit LLVM IR as text (platform-independent)
```

Then compile with `llc` or `clang`:

```
$ llc -march=x86-64 -o output.o module.ll       # x86-64
$ llc -march=aarch64 -o output.o module.ll      # ARM
$ llc -march=nvptx64 -o output.ptx module.ll    # NVIDIA GPU
$ llc -march=amdgcn -o output.o module.ll       # AMD GPU
```

2.4 Use Cases in the JavaScript Ecosystem

1. **DSL compilers**: Build domain-specific languages that compile to native code (query languages, shader languages, config languages)
2. **JIT compilers**: Runtime code generation for interpreters (JavaScript VMs, database query engines)
3. **GPU compute from Node.js**: Generate CUDA/ROCm kernels at runtime without writing C++ wrappers
4. **WebAssembly optimization**: Generate optimized WASM modules from high-level abstractions
5. **Cross-compilation tools**: Build deployment toolchains that target multiple architectures from one codebase

2.5 Why a TypeScript API Instead of C Bindings?

LLVM provides a comprehensive C API (`llvm-c`), and several Node.js bindings exist (`llvm-node`[3], `llvm-bindings`[2], `node-llvmc`[4]). However, FFI bindings have inherent limitations:

1. **Native dependencies**: Require compiling against specific LLVM versions
   - LLVM 15, 16, 17+ are not API-stable
   - Users must install exact LLVM version on their system
   - Cross-compilation becomes complex
2. **Platform fragility**: Native Node.js addons break across Node versions
   - Rebuilding required for Node 18, 20, 22, etc.
   - Electron and Bun compatibility issues
3. **Distribution complexity**: npm packages with native addons need pre-built binaries for every platform
4. **No tree-shaking**: FFI bundles entire LLVM library (hundreds of MB)

2.6 @euriklis/llvm-ir Design Philosophy

This package takes a different approach: **generate LLVM IR as text**. No native bindings, no FFI, no compiled dependencies.

Advantages:

- **Zero dependencies**: Pure TypeScript, runs in Node.js, Bun, Deno, browsers
- **Instant installation**: `npm install @euriklis/llvm-ir` with no compilation step
- **Type-safe API**: Full IntelliSense with TypeScript types
- **Debuggable output**: Inspect generated IR as human-readable text
- **Composable**: Pipe IR to any LLVM tool (`llc`, `opt`, `lli`)
- **Portable**: Same code works on Windows, macOS, Linux, ARM, x86

Trade-offs:

- Cannot use LLVM's optimizer programmatically (must shell out to `opt`)
- Cannot JIT-compile directly (must use `lli` or write IR to file)
- Validation happens at compile-time via external tools, not runtime

For most use cases—code generation, DSL compilers, build tools—these trade-offs are acceptable. The benefits of simplicity, portability, and zero native dependencies far outweigh the limitations.

2.7 Summary

LLVM provides a universal compilation backend. By generating LLVM IR from TypeScript:

- You stay in the JavaScript ecosystem (familiar tools, npm, TypeScript types)
- You unlock native performance on CPUs and GPUs
- You avoid platform-specific build complexity
- You leverage decades of LLVM optimization research

The rest of this handbook shows you how.

Part II

Getting Started with LLVM IR

Chapter 3

Installation and Setup

```
npm install @euriklis/llvm-ir
```

Or with Bun:

```
bun add @euriklis/llvm-ir
```

That's it. No native dependencies, no compilation step, no system LLVM installation required (though you'll want LLVM's `llc` compiler to generate binaries).

Chapter 4

Implementing TypeScript Functions as LLVM IR

This chapter demonstrates how to translate TypeScript logic into LLVM IR using the library's object-based API. Each example shows the TypeScript concept, the generated IR, and the underlying assembly/execution model.

4.1 Example 1: Simple Arithmetic

TypeScript equivalent:

```
function add(a: number, b: number): number {
  return a + b;
}
```

LLVM IR generation:

```
import {
  LLVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  BinaryInst,
  RetInst
} from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("arithmetic");

const addFunc = new LLVMFunction({
  name: "add",
  returnType: LLVMCodegen.types.i32
});

addFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "a"
}));
addFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "b"
}));

const entry = new BasicBlock("entry");
entry.addInstruction(new BinaryInst({
```

```
  name: "result",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: "a",
  rhs: "b"
}));
entry.setTerminator(new RetInst({
  type: LLVMCodegen.types.i32,
  value: "result"
}));

addFunc.addBasicBlock(entry);
codegen.getModule().addFunction(addFunc);

console.log(codegen.toString());
```

Generated LLVM IR:

```
target triple = "x86_64-unknown-linux-gnu"

define i32 @add(i32 %a, i32 %b) {
entry:
  %result = add i32 %a, %b
  ret i32 %result
}
```

Key concepts:

- `LLVMFunction`: Defines function signature
- `Parameter`: Typed function argument (automatically prefixed with `%`)
- `BasicBlock`: Container for instructions (must end with terminator)
- `BinaryInst`: Arithmetic operation (add, sub, mul, etc.)
- `RetInst`: Return value (terminates basic block)

4.2 Example 2: Conditional Logic (if-else)

TypeScript equivalent:

```
function max(a: number, b: number): number {
  if (a > b) {
    return a;
  } else {
    return b;
  }
}
```

LLVM IR generation with control flow:

```
import {
  LLVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  ICmpInst,
  BrInst,
  RetInst
} from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("conditional");
```

```
const maxFunc = new LLVMFunction({
  name: "max",
  returnType: LLVMCodegen.types.i32
});

maxFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "a"
}));
maxFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "b"
}));

const entry = new BasicBlock("entry");
const ifThen = new BasicBlock("if.then");
const ifElse = new BasicBlock("if.else");

// entry: compare a > b
entry.addInstruction(new ICmpInst({
  name: "cmp",
  predicate: LLVMCodegen.predicates.sgt,  // signed greater than
  type: LLVMCodegen.types.i32,
  lhs: "a",
  rhs: "b"
}));
entry.setTerminator(new BrInst({
  condition: "cmp",
  conditionType: LLVMCodegen.types.i1,
  target: ifThen,
  falseTarget: ifElse
}));

// if.then: return a
ifThen.setTerminator(new RetInst({
  type: LLVMCodegen.types.i32,
  value: "a"
}));

// if.else: return b
ifElse.setTerminator(new RetInst({
  type: LLVMCodegen.types.i32,
  value: "b"
}));

maxFunc.addBasicBlock(entry);
maxFunc.addBasicBlock(ifThen);
maxFunc.addBasicBlock(ifElse);
codegen.getModule().addFunction(maxFunc);

console.log(codegen.toString());
```

Generated LLVM IR:

```
define i32 @max(i32 %a, i32 %b) {
entry:
  %cmp = icmp sgt i32 %a, %b
  br i1 %cmp, label %if.then, label %if.else
```

```
if.then:
  ret i32 %a

if.else:
  ret i32 %b
}
```

Key concepts:

- `ICmpInst`: Integer comparison (returns `i1` boolean)
- `BrInst`: Conditional branch (if-else in LLVM)
- Multiple basic blocks: Each path gets its own block
- Control flow graph: `entry` → `if.then` or `if.else`

4.3 Example 3: Loops with PHI Nodes

TypeScript equivalent:

```
function sum(n: number): number {
  let total = 0;
  for (let i = 0; i < n; i++) {
    total += i;
  }
  return total;
}
```

LLVM IR generation with loops:

```
import {
  LLVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  PhiInst,
  ICmpInst,
  BinaryInst,
  BrInst,
  RetInst
} from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("loop");

const sumFunc = new LLVMFunction({
  name: "sum",
  returnType: LLVMCodegen.types.i32
});

sumFunc.addParameter(new Parameter({
  type: LLVMCodegen.types.i32,
  name: "n"
}));

const entry = new BasicBlock("entry");
const loop = new BasicBlock("loop");
const exit = new BasicBlock("exit");

// entry: unconditional jump to loop
```

```
entry.setTerminator(new BrInst({ target: loop }));

// loop: PHI nodes for i and total
loop.addInstruction(new PhiInst({
  name: "i",
  type: LLVMCodegen.types.i32,
  incomingValues: [
    { value: 0, block: entry },
    { value: "i.next", block: loop }
  ]
}));

loop.addInstruction(new PhiInst({
  name: "total",
  type: LLVMCodegen.types.i32,
  incomingValues: [
    { value: 0, block: entry },
    { value: "total.next", block: loop }
  ]
}));

// total.next = total + i
loop.addInstruction(new BinaryInst({
  name: "total.next",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: "total",
  rhs: "i"
}));

// i.next = i + 1
loop.addInstruction(new BinaryInst({
  name: "i.next",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: "i",
  rhs: 1
}));

// cmp = (i.next < n)
loop.addInstruction(new ICmpInst({
  name: "cmp",
  predicate: LLVMCodegen.predicates.slt,
  type: LLVMCodegen.types.i32,
  lhs: "i.next",
  rhs: "n"
}));

// if cmp, continue loop; else exit
loop.setTerminator(new BrInst({
  condition: "cmp",
  conditionType: LLVMCodegen.types.i1,
  target: loop,
  falseTarget: exit
}));

// exit: return total
exit.setTerminator(new RetInst({
```

```
  type: LLVMCodegen.types.i32,
  value: "total"
}));

sumFunc.addBasicBlock(entry);
sumFunc.addBasicBlock(loop);
sumFunc.addBasicBlock(exit);
codegen.getModule().addFunction(sumFunc);

console.log(codegen.toString());
```

Generated LLVM IR:

```
define i32 @sum(i32 %n) {
entry:
  br label %loop

loop:
  %i = phi i32 [ 0, %entry ], [ %i.next, %loop ]
  %total = phi i32 [ 0, %entry ], [ %total.next, %loop ]
  %total.next = add i32 %total, %i
  %i.next = add i32 %i, 1
  %cmp = icmp slt i32 %i.next, %n
  br i1 %cmp, label %loop, label %exit

exit:
  ret i32 %total
}
```

Key concepts:

- `PhiInst`: Merge values from different control flow paths
- Loop structure: `entry` → `loop` → (`loop` or `exit`)
- SSA form maintained: `i`, `i.next`, `total`, `total.next` are all unique
- Back-edge: `loop` can branch to itself

4.4 Type Shortcuts for Cleaner Code

Throughout this handbook, we use type shortcuts for brevity:

```
import { LLVMCodegen } from "@euriklis/llvm-ir";

// Destructure common types
const { i32, i64, float, double, ptr, void: voidType } = LLVMCodegen.
    types;

// Now use directly
const add = new BinaryInst({
  name: "sum",
  opcode: LLVMCodegen.opcodes.add,
  type: i32,  // Instead of LLVMCodegen.types.i32
  lhs: "a",
  rhs: "b"
});
```

These are equivalent to their fully-qualified names:

- `i32` = `LLVMCodegen.types.i32`

- `float` = `LLVMCodegen.types.float`
- `ptr` = `LLVMCodegen.types.ptr`

Part III

Multi-Architecture Compilation

Chapter 5

Compiling LLVM IR for Different CPU Architectures

One of LLVM IR's key strengths is **write once, compile anywhere**. The same IR can target x86-64, ARM, RISC-V, or WebAssembly by changing the compilation flags.

5.1 Target Triples

A target triple specifies the architecture, vendor, and operating system:

```
import { LLVMCodegen } from "@euriklis/llvm-ir";

const codegen = new LLVMCodegen("my_module");
codegen.targetTriple = "x86_64-unknown-linux-gnu";  // x86-64 Linux
codegen.dataLayout = "e-m:e-p270:32:32-i64:64-f80:128-n8:16:32:64-S128
    ";
```

Common triples:

```
x86_64-unknown-linux-gnu        # Linux x86-64
x86_64-apple-darwin             # macOS Intel
aarch64-unknown-linux-gnu       # Linux ARM64 (Raspberry Pi, AWS
    Graviton)
aarch64-apple-darwin            # macOS Apple Silicon (M1/M2/M3)
armv7-unknown-linux-gnueabihf   # ARM 32-bit (Raspberry Pi 3)
riscv64-unknown-linux-gnu       # RISC-V 64-bit
wasm32-unknown-unknown          # WebAssembly 32-bit
wasm64-unknown-unknown          # WebAssembly 64-bit
```

5.2 Compiling with llc

Once you have LLVM IR, compile it with `llc` (LLVM static compiler):

```
# Generate x86-64 assembly
llc -march=x86-64 -filetype=asm -o output.s module.ll

# Generate ARM64 object file
llc -march=aarch64 -filetype=obj -o output.o module.ll

# Generate RISC-V binary
llc -march=riscv64 -filetype=obj -o output.o module.ll
```

```
# Generate WebAssembly
llc -march=wasm32 -filetype=obj -o output.wasm module.ll
```

Or use `clang` to link directly:

```
clang -o program module.ll
./program
```

5.3 Architecture-Specific Code with this Library

The `@euriklis/llvm-ir` package provides architecture-specific code generators:

x86-64 (default):

```
import { LLVMCodegen } from "@euriklis/llvm-ir";
const codegen = new LLVMCodegen("x86_module");
// Targets x86-64 by default
```

ARM/AArch64:

```
import { ARMCodegen } from "@euriklis/llvm-ir";
const codegen = new ARMCodegen("arm_module", true);  // true = 64-bit
// Generates: target triple = "aarch64-unknown-linux-gnu"
```

RISC-V:

```
import { RISCVCodegen } from "@euriklis/llvm-ir";
const codegen = new RISCVCodegen("riscv_module");
// Generates: target triple = "riscv64-unknown-elf"
```

WebAssembly:

```
import { WASMCodegen } from "@euriklis/llvm-ir";
const codegen = new WASMCodegen("wasm_module");
// Generates: target triple = "wasm32-unknown-unknown"
```

All use the same API—only the code generator class changes.

5.4 Data Layout Strings

The data layout string specifies memory alignment, pointer sizes, and endianness:

```
e-m:e-p270:32:32-i64:64-f80:128-n8:16:32:64-S128
```

Breakdown:

- `e`: Little-endian (Intel, ARM). `E` would be big-endian.
- `m:e`: ELF mangling (Linux).
- `p270:32:32`: Pointers in address space 270 are 32-bit with 32-bit alignment.
- `i64:64`: 64-bit integers have 64-bit alignment.
- `f80:128`: 80-bit floats (x87) have 128-bit alignment.
- `n8:16:32:64`: Native integer widths (CPU can operate on these efficiently).
- `S128`: Stack alignment is 128 bits (16 bytes).

Each architecture has its own data layout. The library sets these automatically when you use architecture-specific code generators.

5.5 Cross-Compilation Example

Same TypeScript code, three architectures:

```
import {
  LLVMCodegen,
  ARMCodegen,
  WASMCodegen,
  LLVMFunction,
  BasicBlock,
  RetInst
} from "@euriklis/llvm-ir";

function createAddFunction(codegen) {
  const func = new LLVMFunction({
    name: "add",
    returnType: LLVMCodegen.types.i32
  });
  // ... (same function body for all architectures)
  codegen.getModule().addFunction(func);
}

// x86-64
const x86 = new LLVMCodegen("x86");
createAddFunction(x86);
console.log("=== x86-64 ===");
console.log(x86.toString());

// ARM64
const arm = new ARMCodegen("arm", true);
createAddFunction(arm);
console.log("=== ARM64 ===");
console.log(arm.toString());

// WebAssembly
const wasm = new WASMCodegen("wasm");
createAddFunction(wasm);
console.log("=== WASM ===");
console.log(wasm.toString());
```

Each will have a different target triple, but the function IR is identical. LLVM's backends handle architecture-specific optimizations (register allocation, instruction selection, calling conventions).

Chapter 6

GPU Programming: Same Function Across All Architectures

LLVM supports GPU targets via architecture-specific backends:

- **NVIDIA**: NVPTX backend (CUDA)
- **AMD**: AMDGPU backend (ROCm)
- **Intel/OpenCL**: SPIR-V backend

This chapter shows how to write the same algorithm (vector addition) for all three GPU architectures using this library's unified API.

6.1 Vector Addition: NVIDIA CUDA

CUDA C equivalent:

```
__global__ void vectorAdd(float* a, float* b, float* c, int n) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < n) {
    c[idx] = a[idx] + b[idx];
  }
}
```

TypeScript with NVVMCodegen:

```
import {
  NVVMCodegen,
  LLVMFunction,
  Parameter,
  BasicBlock,
  BinaryInst,
  ICmpInst,
  BrInst,
  LoadInst,
  StoreInst,
  GetElementPtrInst,
  RetInst,
  LLVMCodegen,
  FloatType
} from "@euriklis/llvm-ir";

const codegen = new NVVMCodegen("vector_add");
const module = codegen.getModule();
```

```
const floatType = new FloatType("float");

const kernel = new LLVMFunction({
  name: "vectorAdd",
  returnType: LLVMCodegen.types.void
});

kernel.addParameter(new Parameter({ type: LLVMCodegen.types.ptr, name:
    "a" }));
kernel.addParameter(new Parameter({ type: LLVMCodegen.types.ptr, name:
    "b" }));
kernel.addParameter(new Parameter({ type: LLVMCodegen.types.ptr, name:
    "c" }));
kernel.addParameter(new Parameter({ type: LLVMCodegen.types.i32, name:
    "n" }));

const entry = new BasicBlock("entry");
const compute = new BasicBlock("compute");
const exit = new BasicBlock("exit");

// Get thread ID using CUDA intrinsics
const blockIdx = NVVMCodegen.cuda.blockIdxX("blockIdx.x");
const blockDim = NVVMCodegen.cuda.blockDimX("blockDim.x");
const threadIdx = NVVMCodegen.cuda.threadIdxX("threadIdx.x");

entry.addInstruction(blockIdx);
entry.addInstruction(blockDim);
entry.addInstruction(threadIdx);

// idx = blockIdx.x * blockDim.x + threadIdx.x
const mul = new BinaryInst({
  name: "mul",
  opcode: LLVMCodegen.opcodes.mul,
  type: LLVMCodegen.types.i32,
  lhs: blockIdx,
  rhs: blockDim
});
entry.addInstruction(mul);

const idx = new BinaryInst({
  name: "idx",
  opcode: LLVMCodegen.opcodes.add,
  type: LLVMCodegen.types.i32,
  lhs: mul,
  rhs: threadIdx
});
entry.addInstruction(idx);

// Bounds check: idx < n
const cmp = new ICmpInst({
  name: "cmp",
  predicate: LLVMCodegen.predicates.slt,
  type: LLVMCodegen.types.i32,
  lhs: idx,
  rhs: "n"
});
entry.addInstruction(cmp);
```

```
entry.setTerminator(new BrInst({
  condition: cmp,
  conditionType: LLVMCodegen.types.i1,
  target: compute,
  falseTarget: exit
}));

// Compute: c[idx] = a[idx] + b[idx]
// ... (full implementation in repository examples)

kernel.addBasicBlock(entry);
kernel.addBasicBlock(compute);
kernel.addBasicBlock(exit);
module.addFunction(kernel);

console.log(codegen.toString());
```

Key CUDA intrinsics:

- `NVVMCodegen.cuda.threadIdxX/Y/Z`: Thread index within block
- `NVVMCodegen.cuda.blockIdxX/Y/Z`: Block index within grid
- `NVVMCodegen.cuda.blockDimX/Y/Z`: Block dimensions
- `NVVMCodegen.cuda.syncThreads()`: Barrier synchronization

6.2 Vector Addition: AMD ROCm

HIP equivalent (AMD's CUDA-like API):

```
__global__ void vectorAdd(float* a, float* b, float* c, int n) {
  int idx = hipBlockIdx_x * hipBlockDim_x + hipThreadIdx_x;
  if (idx < n) {
    c[idx] = a[idx] + b[idx];
  }
}
```

TypeScript with AMDGPUCodegen:

```
import { AMDGPUCodegen } from "@euriklis/llvm-ir";

const codegen = new AMDGPUCodegen("vector_add");

// Same structure as CUDA, but use AMD intrinsics:
const workitemId = AMDGPUCodegen.rocm.workitemIdX("tid.x");
const workgroupId = AMDGPUCodegen.rocm.workgroupIdX("gid.x");
// Workgroup size is typically passed as kernel parameter

// Rest identical to CUDA version
```

Key AMD intrinsics:

- `AMDGPUCodegen.rocm.workitemIdX/Y/Z`: Work-item ID (like `threadIdx`)
- `AMDGPUCodegen.rocm.workgroupIdX/Y/Z`: Work-group ID (like `blockIdx`)
- `AMDGPUCodegen.rocm.barrier()`: Work-group barrier

6.3 Vector Addition: Intel GPU (OpenCL)

OpenCL kernel:

```
__kernel void vectorAdd(__global float* a, __global float* b,
                        __global float* c, int n) {
  int idx = get_global_id(0);
  if (idx < n) {
    c[idx] = a[idx] + b[idx];
  }
}
```

TypeScript with SPIRVCodegen:

```
import { SPIRVCodegen } from "@euriklis/llvm-ir";

const codegen = new SPIRVCodegen("vector_add");

// OpenCL provides global ID directly
const globalId = SPIRVCodegen.opencl.getGlobalIdX("gid");
// Note: Returns i64, may need ZExtInst for conversions

// Rest similar to CUDA/AMD versions
```

Key OpenCL intrinsics:

- `SPIRVCodegen.opencl.getGlobalIdX/Y/Z`: Global work-item ID (i64)
- `SPIRVCodegen.opencl.getLocalIdX/Y/Z`: Local work-item ID
- `SPIRVCodegen.opencl.getGroupIdX/Y/Z`: Work-group ID
- `SPIRVCodegen.opencl.barrier()`: Work-group barrier

6.4 GPU Architecture Comparison

Concept	**NVIDIA (NVPTX)**	**AMD (AMDGPU)**	**Intel (SPIR-V)**
Thread ID	`threadIdx`	`workitemId`	`get_local_id`
Block/Group ID	`blockIdx`	`workgroupId`	`get_group_id`
Block/Group Size	`blockDim`	(parameter)	`get_local_size`
Global ID	`blockIdx * blockDim + threadIdx`	(computed)	`get_global_id`
SIMD Width	Warp (32)	Wavefront (64)	Sub-group (8-32)
Synchronization	`__syncthreads`	`barrier`	`barrier`
Shared Memory	`__shared__` (addr space 3)	`__local` (addr space 3)	`__local` (addr space 3)

All three dialects are production-ready with comprehensive intrinsic libraries and `llc`-validated IR generation. See the repository's `src/` directory for complete working examples.

Part IV

Reference

Chapter 7

Instruction Reference

This chapter provides a comprehensive reference for all LLVM IR instructions supported by the library.

7.1 Arithmetic Operations

Integer arithmetic:

Instruction	TypeScript	LLVM IR
Addition	`BinaryInst{opcode: "add"}`	`%r = add i32 %a, %b`
Subtraction	`BinaryInst{opcode: "sub"}`	`%r = sub i32 %a, %b`
Multiplication	`BinaryInst{opcode: "mul"}`	`%r = mul i32 %a, %b`
Division (signed)	`BinaryInst{opcode: "sdiv"}`	`%r = sdiv i32 %a, %b`
Division (unsigned)	`BinaryInst{opcode: "udiv"}`	`%r = udiv i32 %a, %b`
Remainder (signed)	`BinaryInst{opcode: "srem"}`	`%r = srem i32 %a, %b`
Remainder (unsigned)	`BinaryInst{opcode: "urem"}`	`%r = urem i32 %a, %b`

Floating-point arithmetic:

Instruction	TypeScript	LLVM IR
Addition	`BinaryInst{opcode: "fadd"}`	`%r = fadd float %a, %b`
Subtraction	`BinaryInst{opcode: "fsub"}`	`%r = fsub float %a, %b`
Multiplication	`BinaryInst{opcode: "fmul"}`	`%r = fmul float %a, %b`
Division	`BinaryInst{opcode: "fdiv"}`	`%r = fdiv float %a, %b`
Remainder	`BinaryInst{opcode: "frem"}`	`%r = frem float %a, %b`

7.2 Bitwise Operations

Instruction	TypeScript	LLVM IR
Bitwise AND	`BinaryInst{opcode: "and"}`	`%r = and i32 %a, %b`
Bitwise OR	`BinaryInst{opcode: "or"}`	`%r = or i32 %a, %b`
Bitwise XOR	`BinaryInst{opcode: "xor"}`	`%r = xor i32 %a, %b`
Shift left	`BinaryInst{opcode: "shl"}`	`%r = shl i32 %a, %b`
Logical shift right	`BinaryInst{opcode: "lshr"}`	`%r = lshr i32 %a, %b`
Arithmetic shift right	`BinaryInst{opcode: "ashr"}`	`%r = ashr i32 %a, %b`

7.3 Comparison Operations

Integer comparisons (ICmpInst):

Predicate	Meaning	Example
`eq`	Equal	`%r = icmp eq i32 %a, %b`
`ne`	Not equal	`%r = icmp ne i32 %a, %b`
`sgt`	Signed greater	`%r = icmp sgt i32 %a, %b`
`sge`	Signed greater/equal	`%r = icmp sge i32 %a, %b`
`slt`	Signed less	`%r = icmp slt i32 %a, %b`
`sle`	Signed less/equal	`%r = icmp sle i32 %a, %b`
`ugt`	Unsigned greater	`%r = icmp ugt i32 %a, %b`
`uge`	Unsigned greater/equal	`%r = icmp uge i32 %a, %b`
`ult`	Unsigned less	`%r = icmp ult i32 %a, %b`
`ule`	Unsigned less/equal	`%r = icmp ule i32 %a, %b`

Floating-point comparisons (FCmpInst):

Predicate	Meaning	Example
`oeq`	Ordered equal	`%r = fcmp oeq float %a, %b`
`one`	Ordered not equal	`%r = fcmp one float %a, %b`
`ogt`	Ordered greater	`%r = fcmp ogt float %a, %b`
`oge`	Ordered greater/equal	`%r = fcmp oge float %a, %b`
`olt`	Ordered less	`%r = fcmp olt float %a, %b`
`ole`	Ordered less/equal	`%r = fcmp ole float %a, %b`
`uno`	Unordered (NaN)	`%r = fcmp uno float %a, %b`

7.4 Memory Operations

Stack allocation:

```
import { AllocaInst } from "@euriklis/llvm-ir";

const alloc = new AllocaInst({
  name: "ptr",
  allocatedType: LLVMCodegen.types.i32,
  align: 4
});
// Generates: %ptr = alloca i32, align 4
```

Load from memory:

```
import { LoadInst } from "@euriklis/llvm-ir";

const load = new LoadInst({
  name: "val",
  type: LLVMCodegen.types.i32,
  pointerType: LLVMCodegen.types.ptr,
  pointer: "ptr",
  align: 4
});
// Generates: %val = load i32, ptr %ptr, align 4
```

Store to memory:

```
import { StoreInst } from "@euriklis/llvm-ir";

const store = new StoreInst({
  valueType: LLVMCodegen.types.i32,
  value: 42,
  pointerType: LLVMCodegen.types.ptr,
  pointer: "ptr",
  align: 4
});
// Generates: store i32 42, ptr %ptr, align 4
```

Array/struct indexing (GetElementPtr):

```
import { GetElementPtrInst } from "@euriklis/llvm-ir";

const gep = new GetElementPtrInst({
  name: "elem_ptr",
  baseType: LLVMCodegen.types.i32,
  ptrType: LLVMCodegen.types.ptr,
  pointer: "array",
  indices: [
    { type: LLVMCodegen.types.i32, value: 0 },
    { type: LLVMCodegen.types.i32, value: "idx" }
  ]
});
// Generates: %elem_ptr = getelementptr i32, ptr %array, i32 0, i32 %
   idx
```

7.5 Control Flow

Conditional branch:

```
import { BrInst } from "@euriklis/llvm-ir";
```

```
const br = new BrInst({
  condition: "cmp",
  conditionType: LLVMCodegen.types.i1,
  target: thenBlock,
  falseTarget: elseBlock
});
// Generates: br i1 %cmp, label %then, label %else
```

Unconditional branch:

```
const br = new BrInst({ target: nextBlock });
// Generates: br label %next
```

Return:

```
import { RetInst } from "@euriklis/llvm-ir";

// Return value
const ret = new RetInst({
  type: LLVMCodegen.types.i32,
  value: 42
});
// Generates: ret i32 42

// Return void
const retVoid = new RetInst({});
// Generates: ret void
```

Switch statement:

```
import { SwitchInst } from "@euriklis/llvm-ir";

const sw = new SwitchInst({
  value: "x",
  valueType: LLVMCodegen.types.i32,
  defaultDest: defaultBlock,
  cases: [
    { value: 0, dest: case0Block },
    { value: 1, dest: case1Block },
    { value: 2, dest: case2Block }
  ]
});
// Generates: switch i32 %x, label %default [
//              i32 0, label %case0
//              i32 1, label %case1
//              i32 2, label %case2
//            ]
```

7.6 Type Conversions

Instruction	Purpose	Example
`TruncInst`	Truncate integer	`%r = trunc i64 %a to i32`
`ZExtInst`	Zero-extend integer	`%r = zext i32 %a to i64`
`SExtInst`	Sign-extend integer	`%r = sext i32 %a to i64`

FPTruncInst	Truncate float	%r = fptrunc double %a to float
FPExtInst	Extend float	%r = fpext float %a to double
BitCastInst	Reinterpret bits	%r = bitcast i32 %a to float

7.7 Function Calls

```
import { CallInst } from "@euriklis/llvm-ir";

const call = new CallInst({
  name: "result",
  returnType: LLVMCodegen.types.i32,
  functionName: "my_function",
  args: [
    { type: LLVMCodegen.types.i32, value: 10 },
    { type: LLVMCodegen.types.float, value: "x" }
  ]
});
// Generates: %result = call i32 @my_function(i32 10, float %x)
```

7.8 Vector Operations

```
import {
  InsertElementInst,
  ExtractElementInst,
  ShuffleVectorInst,
  VectorType
} from "@euriklis/llvm-ir";

const vec4 = new VectorType({ elementType: float, count: 4 });

// Insert element
const insert = new InsertElementInst({
  name: "v2",
  vectorType: vec4,
  vector: "v1",
  element: 3.14,
  elementType: float,
  index: 0,
  indexType: i32
});
// Generates: %v2 = insertelement <4 x float> %v1, float 3.14, i32 0

// Extract element
const extract = new ExtractElementInst({
  name: "elem",
  vectorType: vec4,
  vector: "v",
  index: 2,
  indexType: i32
});
```

```
// Generates: %elem = extractelement <4 x float> %v, i32 2

// Shuffle vector
const shuffle = new ShuffleVectorInst({
  name: "result",
  vectorType: vec4,
  vector1: "v1",
  vector2: "v2",
  mask: [0, 1, 6, 7]
});
// Generates: %result = shufflevector <4 x float> %v1, <4 x float> %v2,
//                                    <4 x i32> <i32 0, i32 1, i32 6,
    i32 7>
```

7.9 Atomic Operations

```
import { AtomicRMWInst, AtomicCmpXchgInst, FenceInst } from "@euriklis/
    llvm-ir";

// Atomic read-modify-write
const atomicAdd = new AtomicRMWInst({
  name: "old_val",
  operation: "add",
  pointer: "counter",
  pointerType: ptr,
  value: 1,
  valueType: i32,
  ordering: "seq_cst"
});
// Generates: %old_val = atomicrmw add ptr %counter, i32 1 seq_cst

// Compare-and-swap
const cas = new AtomicCmpXchgInst({
  name: "result",
  pointer: "lock",
  pointerType: ptr,
  cmp: 0,
  new: 1,
  valueType: i32,
  successOrdering: "seq_cst",
  failureOrdering: "acquire"
});
// Generates: %result = cmpxchg ptr %lock, i32 0, i32 1 seq_cst acquire

// Memory fence
const fence = new FenceInst({ ordering: "seq_cst" });
// Generates: fence seq_cst
```

Chapter 8

Memory Model and Address Spaces

8.1 Stack vs Heap

LLVM IR supports two memory allocation strategies:

Stack allocation (alloca):

```
const alloc = new AllocaInst({
  name: "local_var",
  allocatedType: i32,
  align: 4
});
// Generates: %local_var = alloca i32, align 4
```

- Automatically deallocated when function returns
- Fast (stack pointer bump)
- Limited size (typical stack: 1-8 MB)
- Good for small, short-lived data

Heap allocation (external malloc):

```
// Declare malloc
const mallocDecl = new LLVMFunction({
  name: "malloc",
  returnType: ptr,
  isExternal: true
});
mallocDecl.addParameter(new Parameter({ type: i64, name: "size" }));

// Call malloc
const heapPtr = new CallInst({
  name: "heap_ptr",
  returnType: ptr,
  functionName: "malloc",
  args: [{ type: i64, value: 4096 }]
});
// Generates: %heap_ptr = call ptr @malloc(i64 4096)
```

- Manual deallocation required (`free`)
- Slower (OS allocator overhead)
- Virtually unlimited size (limited by RAM)
- Good for large, long-lived data

8.2 Alignment

Alignment specifies byte boundaries for memory access:

```
const load = new LoadInst({
  name: "val",
  type: i64,
  pointerType: ptr,
  pointer: "ptr",
  align: 8  // 8-byte alignment for 64-bit integers
});
// Generates: %val = load i64, ptr %ptr, align 8
```

Why alignment matters:

- **x86**: Misaligned access is slow (extra memory cycles)
- **ARM**: Misaligned access may trap (SIGBUS crash)
- **GPU**: Coalesced memory access requires alignment

Common alignments:

Type	Alignment	Reason
`i8`	1 byte	Single byte, no alignment needed
`i16`	2 bytes	16-bit values must be 2-byte aligned
`i32`	4 bytes	32-bit values must be 4-byte aligned
`i64`	8 bytes	64-bit values must be 8-byte aligned
`float`	4 bytes	IEEE 754 single precision (32-bit)
`double`	8 bytes	IEEE 754 double precision (64-bit)
`ptr`	8 bytes (64-bit)	Pointer size depends on architecture
Struct	Largest member	Structs align to their largest field

8.3 Address Spaces (GPU Memory)

GPUs use **address spaces** to distinguish between memory types:

Space	Name	Scope	Performance
0	Generic	All	Compiler chooses
1	Global	All threads	Slow (DRAM)
3	Shared (CUDA/AMD)	Thread block	Fast (on-chip)
4	Constant	All threads	Fast (cached)
5	Local (CUDA)	Single thread	Varies (register spill)

Declaring global memory (address space 1):

```
import { GlobalVariable } from "@euriklis/llvm-ir";

const globalArray = new GlobalVariable({
  name: "global_data",
  type: new ArrayType({ elementType: float, size: 1024 }),
  addressSpace: 1,  // Global memory
  linkage: "external"
});
// Generates: @global_data = external addrspace(1) global [1024 x float
   ]
```

Declaring shared memory (address space 3):

```
const sharedMem = new GlobalVariable({
  name: "shared_tile",
  type: new ArrayType({ elementType: float, size: 256 }),
  addressSpace: 3,  // Shared memory (CUDA __shared__)
  linkage: "internal"
});
// Generates: @shared_tile = internal addrspace(3) global [256 x float]
```

8.4 GetElementPtr (GEP) Operations

`GetElementPtr` computes addresses within aggregates (arrays, structs):

```
// Array indexing: array[idx]
const gep = new GetElementPtrInst({
  name: "elem_ptr",
  baseType: i32,
  ptrType: ptr,
  pointer: "array",
  indices: [{ type: i32, value: "idx" }]
});
// Generates: %elem_ptr = getelementptr i32, ptr %array, i32 %idx

// Struct field access: struct.field1
const structGep = new GetElementPtrInst({
  name: "field_ptr",
  baseType: structType,
  ptrType: ptr,
  pointer: "struct_ptr",
  indices: [
    { type: i32, value: 0 },   // First: dereference pointer
    { type: i32, value: 1 }    // Second: select field 1
  ]
});
// Generates: %field_ptr = getelementptr %StructType, ptr %struct_ptr,
   i32 0, i32 1
```

Important: GEP does *not* load memory—it only computes addresses. Use `LoadInst` to read the value.

8.5 Volatile Access

Mark memory operations as `volatile` to prevent optimization:

```
const load = new LoadInst({
  name: "val",
  type: i32,
  pointerType: ptr,
  pointer: "mmio_reg",
  align: 4,
  isVolatile: true  // Prevent optimization
});
// Generates: %val = load volatile i32, ptr %mmio_reg, align 4
```

Use cases:

- Memory-mapped I/O registers
- Signal handlers
- Multithreaded shared variables (prefer atomics instead)

8.6 Atomic Operations

For thread-safe memory access, use atomic instructions:

```
const atomicLoad = new LoadInst({
  name: "val",
  type: i32,
  pointerType: ptr,
  pointer: "shared_var",
  align: 4,
  atomic: true,
  ordering: "acquire"
});
// Generates: %val = load atomic i32, ptr %shared_var acquire, align 4

const atomicStore = new StoreInst({
  valueType: i32,
  value: 42,
  pointerType: ptr,
  pointer: "shared_var",
  align: 4,
  atomic: true,
  ordering: "release"
});
// Generates: store atomic i32 42, ptr %shared_var release, align 4
```

Memory orderings:

- `unordered`: No synchronization (fastest)
- `monotonic`: Single total order (still weak)
- `acquire`: Synchronize-with release (for loads)
- `release`: Synchronize-with acquire (for stores)
- `acq_rel`: Both acquire and release
- `seq_cst`: Sequentially consistent (safest, slowest)

Chapter 9

Type System Reference

LLVM IR is strongly typed. Every value, instruction, and function has an explicit type.

9.1 Integer Types

Fixed-width integers:

```
import { IntType } from "@euriklis/llvm-ir";

const i1 = LLVMCodegen.types.i1;      // Boolean (1 bit)
const i8 = LLVMCodegen.types.i8;      // Byte (8 bits)
const i16 = LLVMCodegen.types.i16;    // Short (16 bits)
const i32 = LLVMCodegen.types.i32;    // Int (32 bits)
const i64 = LLVMCodegen.types.i64;    // Long (64 bits)
const i128 = LLVMCodegen.types.i128;  // Quad (128 bits)
```

Arbitrary-width integers:

```
const i7 = new IntType({ bits: 7 });    // 7-bit integer
const i24 = new IntType({ bits: 24 });  // 24-bit integer (RGB color)
const i256 = new IntType({ bits: 256 }); // 256-bit integer (crypto)
// Generates: i7, i24, i256
```

LLVM supports integers from `i1` to `i2^23-1` (8 million bits).

9.2 Floating-Point Types

```
import { FloatType } from "@euriklis/llvm-ir";

const half = new FloatType("half");      // f16: 16-bit half precision
const bfloat = new FloatType("bfloat");  // bf16: Brain float (ML)
const float = new FloatType("float");    // f32: Single precision (IEEE
    754)
const double = new FloatType("double");  // f64: Double precision
const fp128 = new FloatType("fp128");    // Quad precision (128-bit)
```

When to use each:

- `half (f16)`: GPU compute, ML inference (low precision, high throughput)
- `bfloat (bf16)`: ML training (Google TPUs, NVIDIA Ampere+)
- `float (f32)`: Default for graphics and general compute
- `double (f64)`: Scientific computing, high-precision math
- `fp128`: Rare (extreme precision needs)

9.3 Pointer Types

In LLVM IR, pointers are opaque (no element type in modern LLVM):

```
const ptr = LLVMCodegen.types.ptr;  // Opaque pointer
// Generates: ptr

// Address space variant
const ptr_addrspace3 = new PointerType(3);  // ptr addrspace(3)
// Generates: ptr addrspace(3) (CUDA shared memory)
```

9.4 Array Types

```
import { ArrayType } from "@euriklis/llvm-ir";

const intArray = new ArrayType({
  elementType: i32,
  size: 10
});
// Generates: [10 x i32]

const floatMatrix = new ArrayType({
  elementType: new ArrayType({ elementType: float, size: 4 }),
  size: 4
});
// Generates: [4 x [4 x float]] (4x4 matrix)
```

9.5 Struct Types

```
import { StructType } from "@euriklis/llvm-ir";

// Named struct
const Point = new StructType({
  name: "Point",
  fields: [
    { type: float },
    { type: float },
    { type: float }
  ]
});
// Generates: %Point = type { float, float, float }

// Packed struct (no padding)
const PackedPoint = new StructType({
  name: "PackedPoint",
  fields: [
    { type: i8 },
    { type: i32 }
  ],
  packed: true
});
// Generates: %PackedPoint = type <{ i8, i32 }>
```

9.6 Vector Types

SIMD vectors for parallel operations:

```
import { VectorType } from "@euriklis/llvm-ir";

// <4 x float> - 4-element float vector (SSE, NEON)
const vec4 = new VectorType({
  elementType: float,
  count: 4
});

// <8 x i32> - 8-element integer vector (AVX)
const vec8i = new VectorType({
  elementType: i32,
  count: 8
});

// Vector addition
const vadd = new BinaryInst({
  name: "vsum",
  opcode: LLVMCodegen.opcodes.fadd,
  type: vec4,
  lhs: "a",
  rhs: "b"
});
// Generates: %vsum = fadd <4 x float> %a, %b
// This adds 4 floats in parallel!
```

9.7 Function Types

```
import { FunctionType } from "@euriklis/llvm-ir";

// int add(int, int)
const addFnType = new FunctionType({
  returnType: i32,
  paramTypes: [i32, i32]
});
// Type: i32 (i32, i32)

// void print(char*)
const printFnType = new FunctionType({
  returnType: voidType,
  paramTypes: [ptr]
});
// Type: void (ptr)

// Variadic: int printf(char*, ...)
const printfFnType = new FunctionType({
  returnType: i32,
  paramTypes: [ptr],
  isVarArg: true
});
// Type: i32 (ptr, ...)
```

9.8 Void Type

Special type for functions that don't return a value:

```
const { void: voidType } = LLVMCodegen.types;

const func = new LLVMFunction({
  name: "doSomething",
  returnType: voidType
});
// Generates: define void @doSomething() { ... }
```

9.9 Type Compatibility

LLVM is strictly typed. You cannot mix types without explicit conversion:

```
// ERROR: Cannot add i32 and i64
const wrong = new BinaryInst({
  opcode: "add",
  type: i32,
  lhs: "x",  // i32
  rhs: "y"   // i64 - type mismatch!
});

// CORRECT: Convert first
const y64 = new ZExtInst({
  name: "y_extended",
  from: "y",
  fromType: i32,
  toType: i64
});
const correct = new BinaryInst({
  opcode: "add",
  type: i64,
  lhs: y64,
  rhs: "x"
});
```

Part V

Advanced Topics

Chapter 10

GPU Memory Hierarchies and Thread Synchronization

GPU programming requires understanding the memory hierarchy and synchronization primitives to achieve high performance.

10.1 The GPU Memory Hierarchy

GPUs have a complex memory hierarchy with vastly different performance characteristics:

Memory Type	Latency	Bandwidth	Use Case
Registers	1 cycle	>10 TB/s	Loop counters, temporaries
Shared/LDS	20-30 cycles	1-2 TB/s	Tile caching, reductions
L1 Cache	50-100 cycles	500 GB/s - 1 TB/s	Automatic caching
L2 Cache	100-200 cycles	200-500 GB/s	Automatic caching
Global/VRAM	200-400 cycles	500 GB/s - 2 TB/s	Large datasets

The key to GPU performance is **minimizing global memory access** by using registers and shared memory.

10.2 Global Memory (Address Space 1)

Global memory is large (GBs) but slow (hundreds of cycles latency):

```
import { GlobalVariable, ArrayType } from "@euriklis/llvm-ir";

const globalData = new GlobalVariable({
  name: "large_array",
  type: new ArrayType({ elementType: float, size: 1048576 }),  // 1M
     floats
  addressSpace: 1,  // Global memory
  linkage: "external"
});
// Generates: @large_array = external addrspace(1) global [1048576 x
    float]
```

Access patterns matter:

- **Coalesced access**: Threads access consecutive addresses → 1 memory transaction

- **Strided access**: Threads access every Nth element → multiple transactions
- **Random access**: Worst case → many transactions

10.3 Shared Memory (Address Space 3)

Shared memory is small (KB) but fast (20-30 cycle latency):

```
const sharedTile = new GlobalVariable({
  name: "tile",
  type: new ArrayType({ elementType: float, size: 256 }),  // 256
     floats (1 KB)
  addressSpace: 3,  // Shared memory
  linkage: "internal"
});
// Generates: @tile = internal addrspace(3) global [256 x float]
```

Why shared memory is 100x faster:

- On-chip SRAM (not external DRAM)
- Shared across threads in the same block
- Perfect for tiling algorithms (reuse data)

10.4 Constant Memory (Address Space 4)

Constant memory is read-only and cached:

```
const constants = new GlobalVariable({
  name: "coefficients",
  type: new ArrayType({ elementType: float, size: 64 }),
  addressSpace: 4,  // Constant memory
  linkage: "internal",
  isConstant: true
});
// Generates: @coefficients = internal addrspace(4) constant [64 x
   float]
```

When to use:

- Filter coefficients (convolution)
- Lookup tables
- Configuration parameters

10.5 Thread Synchronization

When using shared memory, threads must synchronize to avoid race conditions.

Barrier synchronization:

```
import { CallInst } from "@euriklis/llvm-ir";

// CUDA: __syncthreads()
const barrier = new CallInst({
  functionName: "llvm.nvvm.barrier0",
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.nvvm.barrier0()
```

```
// AMD ROCm: barrier()
const barrierAMD = new CallInst({
  functionName: "llvm.amdgcn.s.barrier",
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.amdgcn.s.barrier()

// OpenCL: barrier(CLK_LOCAL_MEM_FENCE)
const barrierOCL = new CallInst({
  functionName: "_Z7barrierj",  // OpenCL barrier
  returnType: voidType,
  args: [{ type: i32, value: 1 }]  // CLK_LOCAL_MEM_FENCE
});
```

Package-provided helper functions:

Instead of manually creating `CallInst` objects, the package provides convenient helper methods:

```
import { NVVMCodegen, AMDGPUCodegen, SPIRVCodegen } from "@euriklis/
    llvm-ir";

// CUDA: Use the helper (recommended)
const barrier = NVVMCodegen.cuda.syncThreads();
// Returns a CallInst configured for llvm.nvvm.barrier0()

// AMD ROCm: Use the helper
const barrierAMD = AMDGPUCodegen.rocm.barrier();
// Returns a CallInst for llvm.amdgcn.s.barrier()

// OpenCL: Use the helper
const barrierOCL = SPIRVCodegen.opencl.barrier();
// Returns a CallInst for _Z7barrierj with correct arguments

// All helpers return CallInst objects ready to add to your basic
    blocks
entry.addInstruction(barrier);
```

These helpers ensure correct intrinsic names and arguments, making your code cleaner and less error-prone.

What barriers do:

1. All threads in the block wait at the barrier
2. No thread proceeds until ALL threads arrive
3. Ensures memory operations before barrier are visible after

Critical rule: Barriers must be in uniform control flow—all threads must execute the same barrier, or deadlock occurs.

```
// WRONG: Conditional barrier (DEADLOCK!)
if (threadIdx.x < 16) {
  barrier();  // Only some threads hit barrier!
}

// CORRECT: All threads hit barrier
barrier();  // Everyone waits
if (threadIdx.x < 16) {
  // Now safe to use synchronized data
```

```
}
```

10.6 Memory Fences

What are memory fences?

In JavaScript/TypeScript, when you write `array[i] = 42;`, it happens immediately and in order. But in GPUs and CPUs, the hardware can reorder memory operations for performance. A **memory fence** is like saying "wait, make sure all pending writes are actually visible before continuing."

Barriers vs. Fences:

Think of them like this:

- **Barrier** (`syncThreads()`): "Everyone stop and wait here. Don't proceed until all threads arrive." (Synchronization + memory ordering)
- **Fence** (`threadfence()`): "Make my writes visible, but don't wait for other threads." (Memory ordering only)

When to use fences:

Use fences for lock-free algorithms and atomic operations where you need memory ordering guarantees but don't want the overhead of waiting for all threads.

Example: GPU memory fences

```
import { FenceInst, CallInst } from "@euriklis/llvm-ir";

// Standard LLVM fence (works on all architectures)
const fence = new FenceInst({
  ordering: "seq_cst"  // Sequential consistency (strongest)
});
// Generates: fence seq_cst

// CUDA threadfence (block-level) - ensures writes visible to block
const fenceBlock = new CallInst({
  functionName: "llvm.nvvm.membar.cta",
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.nvvm.membar.cta()

// CUDA threadfence (global-level) - ensures writes visible globally
const fenceGlobal = new CallInst({
  functionName: "llvm.nvvm.membar.gl",
  returnType: voidType,
  args: []
});
// Generates: call void @llvm.nvvm.membar.gl()
```

Memory ordering levels:

`FenceInst` supports different ordering strengths (from weakest to strongest):

- `"acquire"`: Ensures reads after fence see all writes before
- `"release"`: Ensures writes before fence are visible after
- `"acq_rel"`: Both acquire and release
- `"seq_cst"`: Sequentially consistent (default, safest)

For most use cases, use `"seq_cst"` (sequential consistency). Only use weaker orderings if you understand low-level memory models and need the performance.

10.7 Tiled Matrix Multiplication Example

Now let's see why shared memory matters with a real example: matrix multiplication $C = A \times B$.

Naive version (global memory only):

Each thread computes one element of C:

```
for (int k = 0; k < N; k++) {
  sum += A[row][k] * B[k][col];  // 2N global memory accesses per
     thread
}
```

Problem: **Every iteration reads from global memory** (200-400 cycles each). For $N = 1024$, that's 2048 slow memory accesses per thread.

Tiled version (shared memory):

1. Load a **tile** of A and B into shared memory
2. Synchronize threads (barrier)
3. Compute using fast shared memory
4. Repeat for next tile

```
import { NVVMCodegen, GlobalVariable, ArrayType } from "@euriklis/llvm-
   ir";

const TILE_SIZE = 16;

// Declare shared memory tiles (address space 3)
const tileA = new GlobalVariable({
  name: "tileA",
  type: new ArrayType({
    elementType: new ArrayType({ elementType: float, size: TILE_SIZE })
       ,
    size: TILE_SIZE
  }),
  addressSpace: 3,  // Shared memory
  linkage: "internal"
});

const tileB = new GlobalVariable({
  name: "tileB",
  type: new ArrayType({
    elementType: new ArrayType({ elementType: float, size: TILE_SIZE })
       ,
    size: TILE_SIZE
  }),
  addressSpace: 3,
  linkage: "internal"
});

// Kernel structure (simplified):
// 1. Load tile from A (global) into tileA (shared)
// 2. Load tile from B (global) into tileB (shared)
// 3. barrier() - ensure all loads complete
// 4. Compute using tileA and tileB (fast!)
// 5. barrier() - before loading next tile
// 6. Repeat for all tiles
```

Full implementation with all details (thread indexing, tile loop, PHI nodes) is available in the repository examples.

Performance analysis:
Naive (global only):

- Each thread: $2N$ global loads (400 cycles each)
- Total latency: $\sim 800N$ cycles
- For $N = 1024$: $\sim 819,200$ cycles

Tiled (shared memory):

- Each tile: $TILE\_SIZE^2$ global loads shared across block
- Tile reused $TILE\_SIZE$ times from shared (20 cycles each)
- Total latency: $\sim \frac{800N}{TILE\_SIZE} + 20N$
- For $N = 1024, TILE = 16$: $\sim 71,680$ cycles
- **11x speedup!**

10.8 Key Takeaways

1. **Use shared memory** to cache frequently accessed data
2. **Synchronize with barriers** after loading and before computing
3. **Tile your algorithms** to fit working sets in shared memory
4. **Coalesce global accesses** when loading tiles
5. **Avoid bank conflicts** in shared memory (advanced topic)

The tiled pattern applies to many GPU algorithms: convolutions, FFTs, reductions, sorting.

Chapter 11

Future Work: Apple Silicon GPU Support

11.1 Current State

This package currently supports Apple Silicon **CPUs** via the ARM/AArch64 backend:

```
import { ARMCodegen } from "@euriklis/llvm-ir";

// Target Apple M3/M5 CPU cores
const codegen = new ARMCodegen("apple_module", true);  // 64-bit
codegen.targetTriple = "arm64-apple-macosx";

// Use NEON SIMD for vectorization (128-bit vectors)
const vec4 = new VectorType({ elementType: float, count: 4 });
// Generates efficient ARM NEON instructions
```

This works well for CPU-bound computations with SIMD acceleration.

11.2 The Apple GPU Challenge

Apple M3/M5 chips have powerful GPUs that are excellent for AI and parallel computing. However, unlike NVIDIA and AMD, Apple uses a **proprietary GPU architecture**:

How GPU compilation works:

- **NVIDIA**: Your code → LLVM IR (NVPTX) → PTX → CUDA binary (supported)
- **AMD**: Your code → LLVM IR (AMDGPU) → GCN/RDNA binary (supported)
- **Apple**: Your code → Metal Shading Language[5] → **AIR (proprietary)** → Metal binary (not supported)

The problem: AIR (Apple Intermediate Representation) is based on LLVM IR, but Apple keeps the format **closed and undocumented**[9]. We cannot generate it directly.

11.3 Why This Matters for TypeScript Developers

If you're coming from a JavaScript/TypeScript background, here's an analogy:

- CUDA/AMD are like **open APIs**—you can generate code programmatically (what this package does)
- Metal is like a **proprietary framework**—you must use Apple's tools and pre-built libraries

Think of it like this:

```
// CUDA/AMD (what this package does)
const kernel = generateLLVMIR({
  instructions: [...],  // Full programmatic control
  intrinsics: [...]
});

// Metal (what Apple requires)
const kernel = `
  kernel void myKernel(...) {
    // Must write Metal Shading Language as text
  }
`;
// Then compile with Apple's tools
```

11.4 How PyTorch and TensorFlow Do It

Frameworks like PyTorch don't generate Metal code either. Instead, they use **MPS (Metal Performance Shaders)**[7][8]—Apple's high-level library:

- Apple provides hundreds of **pre-optimized GPU kernels** (matrix multiply, convolution, etc.)[6]
- PyTorch maps operations to these kernels via the **MPSGraph API**
- This works great for standard ML operations
- For custom kernels, you must write `.metal` files by hand[5]

11.5 What We're Waiting For

Apple has two paths to enable this package's Metal support:

Option 1: Open the AIR format specification

If Apple documents how AIR works[9], we could:

```
import { MetalCodegen } from "@euriklis/llvm-ir";

const codegen = new MetalCodegen("my_kernel");
codegen.targetTriple = "air64-apple-macosx";
// Generate AIR directly, just like we do for NVPTX
```

Option 2: Stable Metal Shading Language generation

Alternatively, we could generate `.metal` source files programmatically[10]:

```
const codegen = new MetalCodegen("my_kernel");
const metalSource = codegen.toMetalSource();  // Generate .metal text
// User compiles: xcrun metal -c kernel.metal
```

This is less elegant than LLVM IR generation, but would work.

11.6 Current Workarounds

Until Apple opens AIR or provides better tooling, TypeScript/JavaScript developers targeting Apple GPUs should:

1. **Use high-level frameworks**: PyTorch MPS, TensorFlow, Core ML
2. **Write Metal Shading Language directly**: Create `.metal` files for custom kernels

3. **Use this package for CPU SIMD**: ARM/NEON support works today
4. **Prototype on NVIDIA/AMD**: Develop with this package's GPU support, deploy to Apple via Metal ports

11.7 Unified Memory Advantage

One unique strength of Apple Silicon: **unified memory architecture**.

- CPU and GPU share the same memory pool (16-128 GB)
- No expensive CPU↔GPU copies (unlike NVIDIA)
- Ideal for workflows mixing CPU and GPU work

Once Metal support is added, this will make Apple Silicon extremely competitive for heterogeneous computing workflows.

11.8 Timeline

When AIR becomes public or Apple provides stable LLVM IR → Metal tooling, we will add:

- `MetalCodegen` class matching the API of `NVVMCodegen`, `AMDGPUCodegen`
- Metal-specific intrinsics (`threadgroup` memory, barriers, SIMD-groups)
- Complete examples for Apple GPU programming
- Validation with Apple's Metal compiler

We're monitoring Apple's developer documentation and LLVM commits for any changes. If you have insights into Metal's IR generation, please contribute!

11.9 Community Contributions Welcome

Possible paths forward:

- **Metal Shading Language generator**: Generate `.metal` source text (similar to how we generate LLVM IR text)
- **MPSGraph bindings**: High-level TypeScript API for Metal Performance Shaders
- **AIR reverse-engineering**: Document the format (challenging, legal concerns)
- **Lobby Apple**: Request official AIR documentation or LLVM Metal backend

If you're interested in helping, see the repository's contribution guidelines.

Bibliography

[1] Suyog Sarda, Mayur Pandey. *LLVM Essentials*. Packt Publishing, 2015. An excellent overview of LLVM fundamentals; this handbook extends those concepts to the TypeScript ecosystem and multi-architecture workflows.

[2] ApsarasX. *llvm-bindings: LLVM bindings for Node.js/JavaScript/TypeScript*. GitHub, 2021. Available at: https://github.com/ApsarasX/llvm-bindings

[3] Maël Nison et al. *llvm-node: Node.js bindings for LLVM*. npm package, 2023. Available at: https://www.npmjs.com/package/llvm-node

[4] Cornell Capra. *node-llvmc: JavaScript/TypeScript FFI bindings for the LLVM C API*. GitHub, 2023. Available at: https://github.com/cucapra/node-llvmc

[5] Apple Inc. *Metal Shading Language Specification, Version 4*. Apple Developer Documentation, 2025. Available at: https://developer.apple.com/metal/Metal-Shading-Language-Specification.pdf

[6] Apple Inc. *Metal Documentation*. Apple Developer, 2024. Available at: https://developer.apple.com/metal/

[7] PyTorch Contributors. *MPS backend — PyTorch Documentation*. PyTorch, 2024. Available at: https://docs.pytorch.org/docs/stable/notes/mps.html

[8] Apple Inc. *Accelerated PyTorch training on Mac*. Apple Developer, 2024. Available at: https://developer.apple.com/metal/pytorch/

[9] SamoZ256. *Breaking down Metal's intermediate representation format*. Medium, 2024. Available at: https://medium.com/@samuliak/breaking-down-metals-intermediate-representation-format-41827022489c

[10] Apple Inc. *Metal Programming Guide*. Apple Developer Documentation Archive, 2024. Available at: https://developer.apple.com/library/archive/documentation/Miscellaneous/Conceptual/MetalProgrammingGuide/